

Slick, Pekko & Typed Actors

Pekko & Typed Actors

Gliederung



- 1. Slick
- 2. Das Aktormodell
- 3. Typed Actors: Grundlagen
- 4. Akteur als State Machine
- 5. Hierarchie & Supervision
- 6. Cluster, Remoting & Sharding
- 7. Pekko in Play
- 8. Übung: ein Todo-Akteur-System

Slick

- **Scala Language-Integrated Connection Kit:** typischerer SQL-Zugriff
- **Functional Relational Mapping (FRM)** statt klassischem ORM:
 - Tabellen werden wie **Collections** behandelt (map, filter, flatMap)
- Alle DB-Operationen sind **asynchron** (über Future)
- **Zweistufig:** erst eine Aktion *beschreiben*, dann *ausführen* (``db.run``)

- **TableQuery**: beschreibt die Tabelle, Abfragen mit ``filter``/``map``/``sortBy``
- **DBIOAction**: ``result`` / ``insert`` / ``update``, eine Aktion, keine Ausführung
- **db.run**: führt die Aktion aus, liefert ``Future[...]`

```
val todos = TableQuery[_TODO_TODO_]
```

```
val query  = todos.filter(_._done == false)    // Query (SQL-Beschreibung)
val action = query.result                      // DBIOAction (nichts passiert)
val result: Future[Seq[_TODO_]] = db.run(action) // erst hier läuft SQL
```

Slick

Repository mit Slick

- In der Architektur aus VL07 ändert sich **nur** die infrastructure-Schicht:
- Trait, Service und Controller bleiben unverändert (**Dependency Inversion**)
- Slick kann auch **streamen** (`db.stream``)

```
private[infrastructure] class SlickTodoRepository @Inject() (db: Database)
  extends TodoRepository {
  import slick.jdbc.PostgresProfile.api._

  def findAll: Future[Seq[Todo]] =
    db.run(todos.result)

  def findById(id: Long): Future[Option[Todo]] =
    db.run(todos.filter(_._id === id).result.headOption)
}
```

Das Aktormodell

Das Aktormodell

Was ist Pekko?



- **Pekko:** Toolkit für nebenläufige, verteilte, reaktive Systeme (Apache-Fork von Akka)
- Bausteine:
 - **Aktoren:** Nebenläufigkeit ohne Locks
 - **Pekko Streams:** Datenflüsse mit Backpressure
 - **Cluster / Persistence:** Verteilung und zustandsbehaftete Entitäten
- **Play läuft selbst auf Pekko**
- Umsetzung des Reactive Manifesto: responsive, resilient, elastic, message-driven

Das Aktormodell

Das Aktormodell



- Ein **Aktor** = gekapselter **Zustand** + **Verhalten** + **Mailbox**
- Aktoren kommunizieren **ausschließlich über Nachrichten** (kein shared memory)
- Jeder Aktor verarbeitet seine Nachrichten **nacheinander**
- Als Reaktion auf eine Nachricht kann ein Aktor:
 - Nachrichten an andere Aktoren senden
 - neue Kind-Aktoren erzeugen
 - sein Verhalten für die nächste Nachricht festlegen

Das Aktormodell

Aktoren vs. Threads



- **Threads + shared state:** mehrere Threads greifen auf dieselben Daten zu
 - Schutz nötig über `synchronized`/Locks, daher Deadlocks und Race Conditions
 - schwer zu testen, nicht-deterministische Fehler
- **Aktoren:** Zustand gehört genau einem Aktor, Zugriff nur per Nachricht
 - kein gemeinsamer Speicher, also keine Locks nötig
 - ein Aktor ist kein Thread: viele Aktoren teilen sich wenige Threads

Das Aktormodell

Mailbox & Dispatcher

- Jeder Aktor hat eine **Mailbox**, eine Warteschlange für eingehende Nachrichten
- Der **Dispatcher** ist ein Thread-Pool, der Aktoren Threads zuteilt:
 - nimmt einen Aktor mit Nachrichten, gibt ihm kurz einen Thread
 - der Aktor verarbeitet einige Nachrichten, dann ist der nächste dran
- So bedienen **wenige Threads sehr viele Aktoren** (kooperatives Scheduling)
- **Wichtig:** im Aktor **niemals blockieren**, sonst belegt man einen Pool-Thread

Das Aktormodell

Warum Aktoren?



- **Nebenläufigkeit ohne Locks:** der Aktor serialisiert Zugriffe auf seinen Zustand
- **Resilience:** Supervision, Eltern überwachen Kinder, "let it crash"
- **Location Transparency:** gleiches Modell lokal und über Knoten verteilt
- **Skalierung:** Millionen leichtgewichtige Aktoren (ca. 2,5 Millionen pro GB Heap) auf wenigen Threads
- Ideal für: zustandsbehaftete Domänen, Echtzeit, IoT, Spiele, Workflows

- **Zustand bleibt im Aktor** und wird nie nach außen gereicht (keine Referenz auf internen State)
- **Niemals blockieren:** kein ``Await``, kein ``Thread.sleep``, kein blockierendes I/O
 - stattdessen ``Future`` + ``pipeToSelf`` (gleich)
- **Nur über Nachrichten** kommunizieren, keine Methodenaufrufe auf andere Aktoren
- **Nachrichten immutabel** halten (keine veränderbaren Objekte verschicken)
- ``ActorContext`` **nicht nach außen leaken** (nur synchron im Aktor verwenden)

Typed Actors: Grundlagen

Typed Actors: Grundlagen

Nachrichtenprotokoll

- **Typed**: ein Akteur akzeptiert nur Nachrichten **eines** Typs
- Protokoll als **geschlossenes** `sealed trait`, Nachrichten **immutabel**:
- `replyTo: ActorRef[T]` = wohin die Antwort geschickt wird (typesicher!)

```
import org.apache.pekko.actor.typed.{ActorRef, Behavior}
```

```
object TodoCache {  
  sealed trait Command  
  case class Add(todo: Todo) extends Command  
  case class GetById(id: Long, replyTo: ActorRef[Option[Todo]]) extends Command  
  case class GetAll(replyTo: ActorRef[Seq[Todo]]) extends Command  
}
```

Typed Actors: Grundlagen

Behavior definieren

- Das **Behavior** beschreibt, wie der Akteur auf Nachrichten reagiert:

```
def apply(): Behavior[Command] = cache(Map.empty)

private def cache(todos: Map[Long, Todo]): Behavior[Command] =
  Behaviors.receiveMessage {
    case Add(todo) =>
      cache(todos + (todo.id -> todo))    // neues Behavior mit neuem State
    case GetById(id, replyTo) =>
      replyTo ! todos.get(id)
      Behaviors.same
    case GetAll(replyTo) =>
      replyTo ! todos.values.toSeq
      Behaviors.same
  }
```

Typed Actors: Grundlagen

State über Behavior-Wechsel

- Aktoren haben **keinen** `var`, Zustand ist ein **Parameter** des Behaviors
- Ein "Zustandswechsel" = ein **neues Behavior** zurückgeben:
- Funktionaler Stil: unveränderlicher State, kein gemeinsamer Speicher
- Da nur eine Nachricht zur Zeit läuft, ist der Wechsel **race-frei**

```
private def cache(todos: Map[Long, Todo]): Behavior[Command] =  
  Behaviors.receiveMessage {  
    case Add(todo) => cache(todos + (todo.id -> todo)) // rekursiv, neuer State  
    // ...  
  }
```

Typed Actors: Grundlagen

Behaviors-Bausteine



- **Behaviors.receiveMessage:** reagiert nur auf die Nachricht
- **Behaviors.receive:** bekommt zusätzlich den `ActorContext` (spawn, log, self)
- **Behaviors.setup:** Initialisierung beim Start (z.B. Kinder erzeugen)
- **Behaviors.same:** Verhalten unverändert lassen
- **Behaviors.stopped:** den Aktor beenden

```
def apply(): Behavior[Command] = Behaviors.setup { context =>
  context.log.info("TodoCache gestartet")
  cache(Map.empty)
}
```

Typed Actors: Grundlagen

Timers: zeitgesteuerte Nachrichten



- Ein Akteur kann sich **selbst** Nachrichten zeitgesteuert schicken:
- ``startSingleTimer(msg, delay)`` = einmalig
- ``startTimerWithFixedDelay(msg, interval)`` = periodisch
- Ideal für: Caches aufräumen, Heartbeats, Timeouts, Polling

```
def apply(): Behavior[Command] = Behaviors.withTimers { timers =>
  timers.startTimerWithFixedDelay(Cleanup, 1.minute) // periodisch
  cache(Map.empty)
}
```

Typed Actors: Grundlagen

Stash: Nachrichten zwischenspeichern

- Manchmal kann ein Akteur eine Nachricht **noch nicht** verarbeiten (z.B. beim Laden)
- **Stash**: Nachrichten zurücklegen und später erneut zustellen:
- Typisch beim **Hochfahren**: erst Daten laden, eingehende Befehle puffern

```
Behaviors.withStash(capacity = 100) { buffer =>
  def initializing: Behavior[Command] =
    Behaviors.receiveMessage {
      case Ready(data) => buffer.unstashAll(running(data)) // alle nachholen
      case other       => buffer.stash(other); Behaviors.same
    }
  initializing
}
```

Typed Actors: Grundlagen

ActorContext: self, log, spawn

- Mit `Behaviors.receive` bekommt man zusätzlich den **ActorContext**:
- `context.self` = die eigene `ActorRef`
- `context.spawn(...)` erzeugt Kind-Aktoren; `context.log` ist strukturiertes Logging

```
Behaviors.receive[Command] { (context, message) =>
  message match {
    case Add(todo) =>
      context.log.info(s"Add ${todo.id}")    // Logging (SLF4J)
      val worker = context.spawn(Worker(), s"worker-${todo.id}")
      worker ! Worker.Process(todo)
      Behaviors.same
  }
}
```

Typed Actors: Grundlagen

ActorSystem & tell

- Ein **ActorSystem** ist der Wurzel-Aktor und die Laufzeitumgebung:
- `!` (**tell**): sendet eine Nachricht, **ohne** auf Antwort zu warten
- Die Nachricht landet in der Mailbox und wird asynchron verarbeitet

```
val cache: ActorSystem[TodoCache.Command] =  
    ActorSystem(TodoCache(), "todo-cache")
```

```
cache ! TodoCache.Add(Todo(1, "Lernen", false)) // "tell"
```

Typed Actors: Grundlagen

ask-Pattern: Request-Response

- Wenn man eine **Antwort** braucht: `ask` (`?`) liefert ein `Future`:
- Pekko erzeugt einen temporären Akteur für `replyTo` und füllt das `Future`
- **Timeout** nötig, weil eine Antwort ausbleiben kann

```
import org.apache.pekko.actor.typed.scaladsl.AskPattern._  
import org.apache.pekko.util.Timeout  
import scala.concurrent.duration._
```

```
given Timeout = 3.seconds  
given Scheduler = cache.scheduler
```

```
val all: Future[Seq[Todo]] =  
  cache.ask(replyTo => TodoCache.GetAll(replyTo))
```

Typed Actors: Grundlagen

tell vs. ask: wann was?

- **tell** (!): feuern und vergessen, kein Rückgabewert, vollständig entkoppelt
 - der **Normalfall** für Kommunikation *zwischen* Aktoren
- **ask** (?): Request-Response, liefert ein `Future`, braucht `Timeout` + `Scheduler`
 - erzeugt intern einen temporären Akteur für die Antwort
 - vor allem an der **Grenze** sinnvoll (z.B. Controller fragt Akteur)
- **Faustregel:** innerhalb des Systems `tell` + eigenes `replyTo`; `ask` nur am Rand

- Ein Akteur selbst soll **nicht blockieren** und auch nicht `Future.ask` abwarten
- `context.ask` schickt die Antwort als **eigene Nachricht** zurück in die Mailbox:
- Das Ergebnis wird in den **eigenen** Command-Typ übersetzt (`MarkAvailable`, ...)
- Ähnlich: `context.messageAdapter`, um fremde Nachrichten in eigene zu wandeln

```
context.ask(inventory, ref => Inventory.Check(productId, ref)) {  
  case Success(Inventory.InStock)    => MarkAvailable(productId)  
  case Success(Inventory.OutOfStock) => MarkUnavailable(productId)  
  case Failure(ex)                   => CheckFailed(productId, ex)  
}
```

Typed Actors: Grundlagen

Futures im Akteur: pipeToSelf

- Service-/DB-Aufrufe liefern ein `Future`, der Akteur darf aber **nicht warten**
- `context.pipeToSelf` leitet das Future-Ergebnis als **Nachricht an sich selbst**:
- Kein Blockieren, kein direkter Zugriff auf den State aus dem Future-Callback

```
case Load(id) =>
  context.pipeToSelf(repository.findById(id)) {
    case Success(todo) => Loaded(todo)      // interne Command-Nachrichten
    case Failure(ex)   => LoadFailed(ex)
  }
Behaviors.same
```

```
case Loaded(todo)    => /* State aktualisieren */ Behaviors.same
case LoadFailed(ex)  => /* Fehler behandeln */   Behaviors.same
```

Typed Actors: Grundlagen

Nachrichten: Ordering & Delivery



- **Delivery:** Pekko garantiert **at-most-once**, eine Nachricht kommt höchstens einmal an
 - kann verloren gehen (besonders über das Netz), keine automatischen Retries
- **Ordering:** Nachrichten **von A nach B** behalten ihre Reihenfolge
 - aber: keine Garantie über die Reihenfolge zwischen *verschiedenen* Sendern
- Wer "genau einmal" braucht, baut **Acknowledgements / Idempotenz** selbst

Aktor als State Machine

Aktor als State Machine

Idee: jeder Zustand ein Behavior



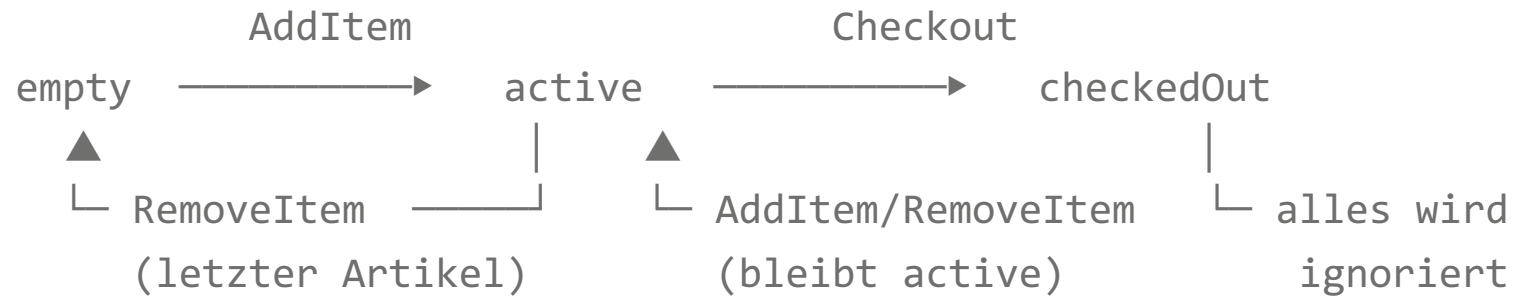
- Ein Aktor ist eine natürliche **endliche Zustandsmaschine** (FSM)
- **Jeder Zustand** = ein eigenes `Behavior`
- Ein **Übergang** = die Rückgabe eines *anderen* Behaviors
- Jeder Zustand kann **eigene Daten** tragen (als Behavior-Parameter)
- **Ungültige Übergänge** existieren gar nicht erst, im Zustand nicht implementiert

- **Naiver Ansatz:** ein `status`-Feld + überall `if (status == ...)` prüfen
 - jeder Befehl muss alle Status kennen, leicht etwas zu vergessen
 - ungültige Kombinationen sind möglich und müssen manuell abgefangen werden
- **FSM-Aktor:** der *Zustand* bestimmt, welche Nachrichten überhaupt behandelt werden
 - Code ist nach **Zuständen** organisiert, nicht nach Befehlen
 - die Struktur erzwingt die erlaubten Übergänge

Aktor als State Machine

Beispiel: Shopping-Cart

- Drei Zustände, Übergänge ausgelöst durch Nachrichten:
- ``empty`` und ``checkedOut`` tragen keine bzw. finale Daten, ``active`` hält die Artikel
- Realer Code: das ``cart``-Feature im Projekt (Modelle ``CartItem``, ``CartState``)



Aktor als State Machine

Cart: Protokoll & Startzustand

```
object Cart {  
  
  def apply(cartId: String): Behavior[Command] = empty(cartId)  
  
  private def empty(cartId: String): Behavior[Command] =  
    Behaviors.receiveMessage {  
      case AddItem(item, replyTo) =>  
        val items = Map(item.productId -> item)  
        replyTo ! CartState.from(cartId, "active", items.values)  
        active(cartId, items) // Übergang empty -> active  
      case Get(replyTo) =>  
        replyTo ! CartState.from(cartId, "empty", Nil); Behaviors.same  
      case _ => Behaviors.same // Remove/Checkout: nichts zu tun  
    }  
}
```

```
private def active(cartId: String, items: Map[String, CartItem]): Behavior[Command] =
  Behaviors.receiveMessage {
    case AddItem(item, replyTo) =>
      val updated = items.updatedWith(item.productId) {
        case Some(e) => Some(e.copy(quantity = e.quantity + item.quantity))
        case None    => Some(item)
      }
      replyTo ! CartState.from(cartId, "active", updated.values)
      active(cartId, updated) // Daten ändern, Zustand bleibt
  }
```

Aktor als State Machine

Cart: aktiver Zustand mit Daten

```
Behaviors.receiveMessage {  
  ...  
  case RemoveItem(id, replyTo) =>  
    val rest = items - id  
    if (rest.isEmpty) { replyTo ! CartState.from(cartId, "empty", Nil); empty(cartId) }  
    else { replyTo ! CartState.from(cartId, "active", rest.values); active(cartId, rest) }  
  
  case Checkout(replyTo) =>  
    replyTo ! CartState.from(cartId, "checkedOut", items.values)  
    checkedOut(cartId, items) // Übergang active -> checkedOut  
}
```

- Nach `Checkout` ist der Warenkorb **eingefroren**, modifizierende Befehle ändern nichts
- Alternativ: statt stillem Ignorieren eine abweisende Fehler-Antwort schicken

```
private def checkedOut(cartId: String, items: Map[String, CartItem]): Behavior[Command] =  
  Behaviors.receiveMessage { msg =>  
    val frozen = CartState.from(cartId, "checkedOut", items.values)  
    msg match {  
      case Get(replyTo)           => replyTo ! frozen  
      case AddItem(_, replyTo)    => replyTo ! frozen    // ignoriert  
      case RemoveItem(_, replyTo) => replyTo ! frozen    // ignoriert  
      case Checkout(replyTo)      => replyTo ! frozen    // ignoriert  
    }  
    Behaviors.same // bleibt immer checkedOut  
  }  
}
```

- Pro `cartId` genau ein Aktor, eine Registry hält die `ActorRef`s:
- Im Controller per `ask`, das Ergebnis ist ein `Future[CartState]`:

@Singleton

```
class CartRegistry @Inject() (system: ActorSystem) {  
    private val carts = new ConcurrentHashMap[String, ActorRef[Cart.Command]]()  
    def cartRef(id: String): ActorRef[Cart.Command] =  
        carts.computeIfAbsent(id, key => system.spawn(Cart(key), s"cart-$key"))  
}  
  
def addItem(cartId: String) = Action.async(parse.json[CartItem]) { req =>  
    registry.cartRef(cartId).ask(Cart.AddItem(req.body, _)).map(s => Ok(Json.toJson(s)))  
}
```

Aktor als State Machine

Vorteile der FSM-Aktoren



- **Übergänge explizit:** jeder Zustand definiert nur die erlaubten Reaktionen
- **Illegale Zustände unmöglich:** verbotene Befehle werden im Zustand gar nicht behandelt
- **Daten pro Zustand:** jeder Behavior trägt genau die Daten, die er braucht
- **Lesbar:** der Code spiegelt das Zustandsdiagramm 1:1 wider
- **Testbar:** jeder Zustand isoliert prüfbar (Nachricht rein, Übergang prüfen)
- Typische Fälle: Warenkorb, Bestell-/Zahlungs-Workflows, Verbindungen, Protokolle
- Mit **Persistence** (Event Sourcing) wird daraus ein langlebiger Workflow

Hierarchie & Supervision

Hierarchie & Supervision

Kind-Aktoren & Hierarchie

- Aktoren bilden eine **Baum-Hierarchie**, jeder Aktor hat einen Eltern-Aktor
- Über den `ActorContext` erzeugt man **Kind-Aktoren** mit `spawn`:
- Das ActorSystem ist die Wurzel ("user guardian"), darunter hängt alles

```
def apply(): Behavior[Command] = Behaviors.setup { context =>
  val worker: ActorRef[Worker.Task] =
    context.spawn(Worker(), "worker")

  Behaviors.receiveMessage { msg =>
    worker ! Worker.Task(msg)    // Arbeit an Kind delegieren
    Behaviors.same
  }
}
```

Hierarchie & Supervision

Supervision & "let it crash"



- Wirft ein Akteur eine Exception, entscheidet die **Supervision-Strategie**:
 - **restart**: Akteur neu starten (State wird zurückgesetzt)
 - **resume**: Fehler ignorieren, weitermachen
 - **stop**: Akteur beenden
- Philosophie **"let it crash"**: Fehler nicht überall abfangen, sondern durch Neustart heilen

```
import org.apache.pekko.actor.typed.SupervisorStrategy
```

```
val supervised: Behavior[Worker.Task] =  
  Behaviors  
    .supervise(Worker())  
    .onFailure[Exception](SupervisorStrategy.restart)
```

Hierarchie & Supervision

Death Watch



- Ein Akteur kann einen anderen **überwachen** und wird bei dessen Ende benachrichtigt:
- Pekko schickt dem Beobachter ein **`Terminated`**-Signal
- Nützlich, um auf das Ende abhängiger Akteure zu reagieren (Neustart, Aufräumen)

```
context.watch(worker)    // überwachen
```

```
Behaviors.receiveSignal {  
  case (context, Terminated(ref)) =>  
    context.log.warn(s"$ref ist beendet")  
    Behaviors.same  
}
```

Hierarchie & Supervision

Lebenszyklus & Signale



- Neben Nachrichten empfängt ein Akteur **Signale** zum Lebenszyklus:
 - `PostStop`: der Akteur wurde gestoppt (Ressourcen freigeben)
 - `PreRestart`: vor einem Neustart
- Ideal zum Aufräumen: Verbindungen schließen, Puffer leeren

```
Behaviors.receiveMessage[Command] { /* ... */ }  
  .receiveSignal {  
    case (context, PostStop) =>  
      context.log.info("TodoCache gestoppt")  
      Behaviors.same  
  }
```

Cluster, Remoting & Sharding

Cluster, Remoting & Sharding

Von lokal zu verteilt



- **Location Transparency:** dasselbe Programmiermodell lokal **und** über das Netz
- Eine `ActorRef` kann auf einen Akteur auf einem **anderen Knoten** zeigen
- Nachrichten werden serialisiert und über das Netzwerk geschickt
- Drei aufeinander aufbauende Schichten:
 - **Remoting:** Transport zwischen JVMs
 - **Cluster:** mehrere Knoten zu einem logischen System
 - **Sharding:** zustandsbehaftete Akteure über den Cluster verteilen

Cluster, Remoting & Sharding

Persistence & Event Sourcing

- Aktor-State ist normalerweise **flüchtig**, bei Neustart/Verschiebung weg
- **Event Sourcing**: nicht den Zustand speichern, sondern die **Ereignisse**
- Beim Start wird der State aus den Events **wiederhergestellt** (Replay)
- Perfekte Ergänzung zu Sharding: Entitäten überleben Knoten-Ausfälle

```
EventSourcedBehavior[Command, Event, State](  
  persistenceId = PersistenceId.ofUniqueId("todo-42"),  
  emptyState    = State.empty,  
  commandHandler = (state, cmd) => ...,  
  eventHandler   = (state, evt) => ...  
)
```

Pekko in Play

Pekko in Play

Aktoren in Play nutzen

- Play stellt ein **ActorSystem** per DI bereit, Typed-Aktor damit `spawn`en:
- Im Controller per `ask` lesen (gibt ein `Future[Result]`):

```
import org.apache.pekko.actor.typed.scaladsl.adapter._
```

```
@Singleton
```

```
class TodoCacheProvider @Inject() (system: ActorSystem) {  
  val cache: ActorRef[TodoCache.Command] =  
    system.spawn(TodoCache(), "todo-cache")  
}
```

```
def cached = Action.async {  
  cache.ask(TodoCache.GetAll(_)).map(todos => Ok(Json.toJson(todos)))  
}
```

Pekko in Play

Aktoren testen

- Pekko bringt ein **ActorTestKit** mit (asynchron, echte Aktoren):
- **TestProbe** ist ein Test-Aktor, der empfangene Nachrichten prüfbar macht
- Für reine Logik: `BehaviorTestKit` (synchron, ohne echtes ActorSystem)

```
val testKit = ActorTestKit()
val probe   = testKit.createTestProbe[Seq[Todo]]()
val cache   = testKit.spawn(TodoCache())
```

```
cache ! TodoCache.Add(Todo(1, "Test", false))
cache ! TodoCache.GetAll(probe.ref)
```

```
probe.expectMessage(Seq(Todo(1, "Test", false)))
```

Übung

- Wir bauen Aktoren **inkrementell** in das Play-Projekt ein
- 1. Command-Protokoll für `TodoCache`
- 2. Behavior mit State (`Map`)
- 3. Aktor in Play spawnen
- 4. Lese-Endpunkt mit `ask`
- 5. Schreiben per `tell`
- 6. Timer: periodisches Logging/Aufräumen
- 7. Workflow-Aktor als **State Machine**
- 8. Supervision & `PostStop`

- 1. `TodoCache`-Objekt mit `sealed trait Command`:
 - `Add(todo)`, `GetAll(replyTo)`, `GetById(id, replyTo)`
- 2. `Behavior[Command]` mit `cache(todos: Map[Long, Todo])`:

```
private def cache(todos: Map[Long, Todo]): Behavior[Command]
```

- `@Singleton`-Provider mit injiziertem `ActorSystem`
- `system.spawn(TodoCache(), "todo-cache")`:

`@Singleton`

```
class TodoCacheProvider @Inject() (system: ActorSystem) {  
    val cache: ActorRef[TodoCache.Command] = ???  
}
```

- 4. Lese-Endpunkt `GET /v1/cache/todos` mit `ask`:
- 5. Beim Anlegen eines Todos zusätzlich `cache ! TodoCache.Add(todo)` (tell)
- given `Timeout` und given `Scheduler` für `ask` bereitstellen

```
def cached = Action.async {  
  cache.ask... ???  
}
```

- Den Cache periodisch aufräumen oder Statistik loggen:
- Neue interne Nachricht `case object LogStats extends Command`
- Im Behavior: `case LogStats => context.log.info(...); Behaviors.same`

```
def apply(): Behavior[Command] = Behaviors.withTimers { timers =>
  timers.startTimerWithFixedDelay(LogStats, 30.seconds)
  ???
}
```

- Einen **Workflow-Aktor** für den Todo-Lebenszyklus bauen:
 - Zustände: `open`, `inProgress`, `done`
 - Nachrichten: `Start`, `Finish`, `Status(replyTo)`
- **Jeder Zustand** ist ein eigenes `Behavior`
- Testen: in `open` ein `Finish` schicken, bleibt `open`

- Todo Manager Akteur welcher die Workflow Akteure als „Children“ spawn und Referenzen dazu hält
- Den `TodoCache` gegen Fehler absichern und sauber beenden:
- Zusätzlich `PostStop` behandeln:
 - ``receiveSignal { case (ctx, PostStop) => ctx.log.info("gestoppt") }`
- Damit überlebt der Akteur Fehler (Neustart) und meldet sein Ende